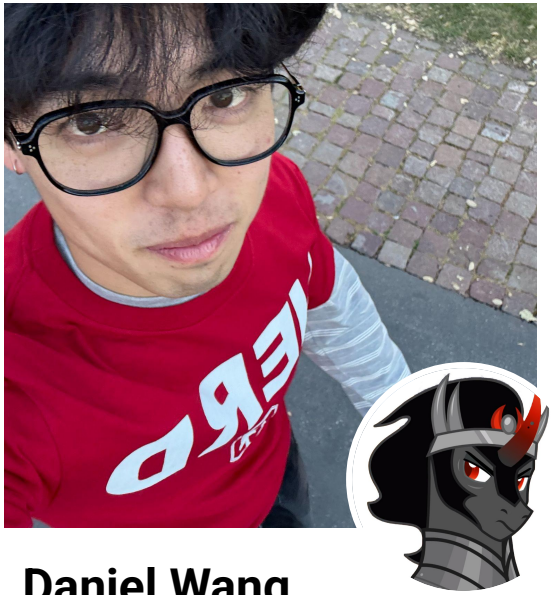
Introductions

Ask us anything right now!

pollev.com/cs61b



Daniel Wang
4th year, CS Major
Favorite non-CS class:
MEDIAST 168
Cybernetics, cybercultures, and cyborgs (w/ Matthew Berry)

Dawn Schumacher
3rd year, Applied Math major
Favorite non-CS class:
Data 104
(unfairly hated!)

Lecture 26

Hash Tables

CS61B, Spring 2026 @ UC Berkeley

Dawn Schumacher and Daniel Wang

Slides Credit: Josh Hug

Motivation, Set Implementations

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

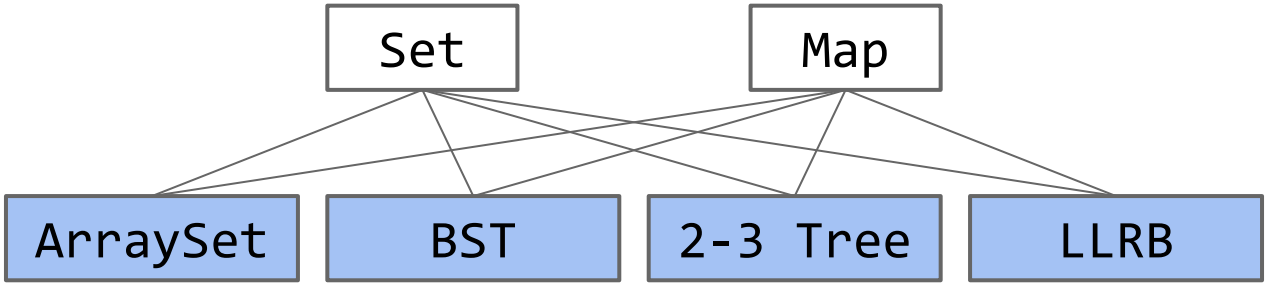
Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

Sets

We've now seen several implementations of the Set (or Map) ADT.



Worst case runtimes

	contains(x)	add(x)	Notes
ArraySet	Θ(N)	Θ(N)	
BST	Θ(N)	Θ(N)	Random trees are Θ(log N).
2-3 Tree	Θ(log N)	Θ(log N)	Beautiful idea. Very hard to implement.
LLRB	Θ(log N)	Θ(log N)	Maintains bijection with 2-3 tree. Hard to implement.

Limits of Search Tree Based Sets

Our search tree based sets require items to be comparable.

- Need to be able to ask “is X < Y?” Not true of all types.
- Could we somehow avoid the need for objects to be comparable?

Our search tree sets have excellent performance, but could maybe be better?

- Θ(log N) is amazing. 1 billion items is still only height ~30.

$$\log_2(1\,000\,000\,000) = 29.897352854$$

- Could we somehow do better than Θ(log N)?

Today we’ll see the answer to both of the questions above is yes.

ArrayOfListsSet

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- **ArrayOfListsSet**
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

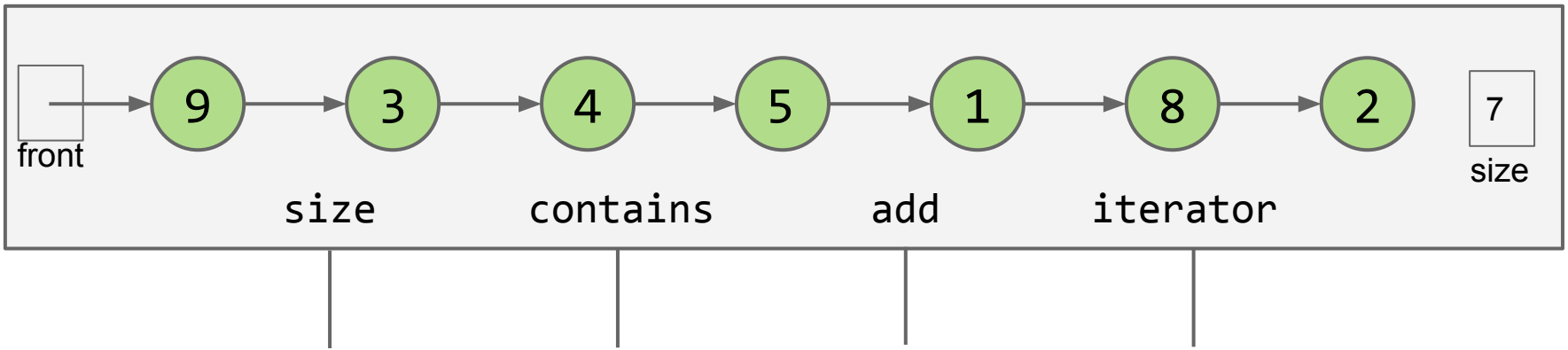
Creating a Good Hash Code (extra)

Linear Probing (extra)

Back to LinkedList

Suppose we have a `LinkedListSet<Integer>`. Let's assume there is no specific order.

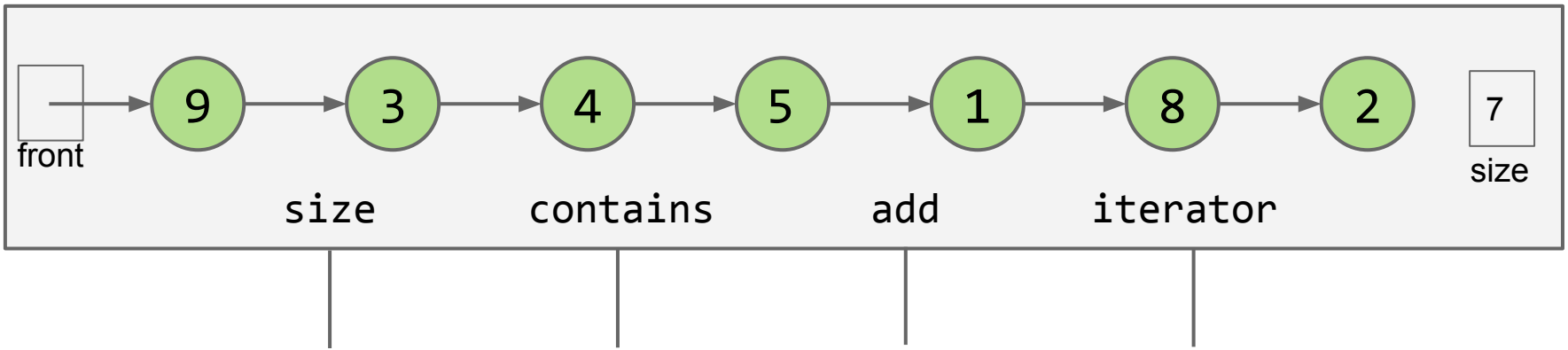
- Contains and add are slow.



Improvements

We considered two different ways we could improve the data structure below.

- Add express lanes (skip lists, out of scope in 61B).
- Transform into a tree like structure (BST and its cousins).



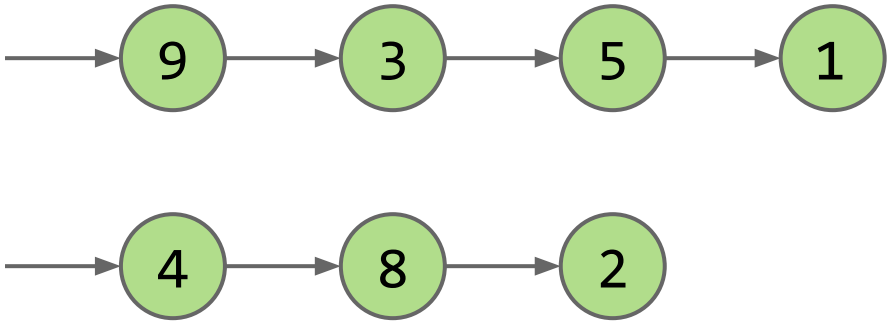
The two approaches above required the data to be in sorted order.

- Our next approach will not.

Key Idea: Multiple Lists

What if we split our list into two lists, one containing the even items, and one containing the odd items?

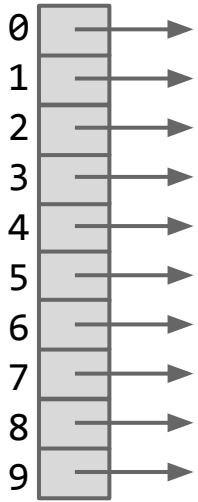
- add and contains operation will be twice as fast if items are evenly distributed.



Even More Lists

Suppose we now have 10 buckets, each containing a list.

- *Note: This paradigm is called "separate chaining". This may seem familiar!*
- How do we decide which integers go in which list?



Even More Lists

When you order from TP Tea on Snackpass, how do you determine which of the slots your drink will end up in?

Even More Lists

When you order from TP Tea on Snackpass, how do you determine which of the slots your drink will end up in?

You look at the last digit!

- Importantly, this is **equivalent to modulus by 10.**

Modulus Review

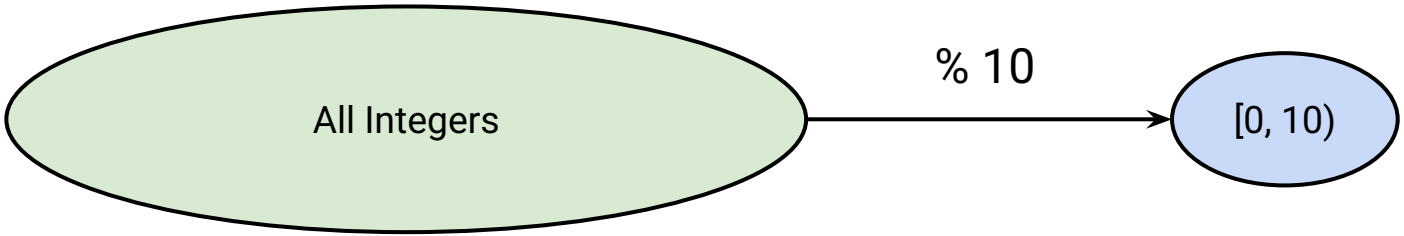
When you take some integer modulo **M**, it's equivalent to the remainder if you were to divide that integer by **M**.

13 % 10 = 3

7 % 10 = 7

1000 % 10 = 0

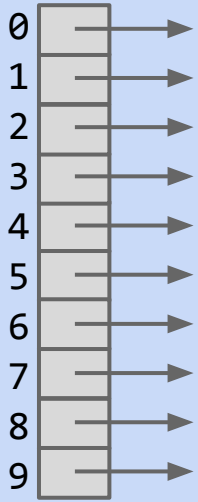
We can think of this as a "reduction" from the space of all integers to the integers from 0 (inclusive) to n (exclusive)



Even More Lists

Suppose we now have 10 lists.

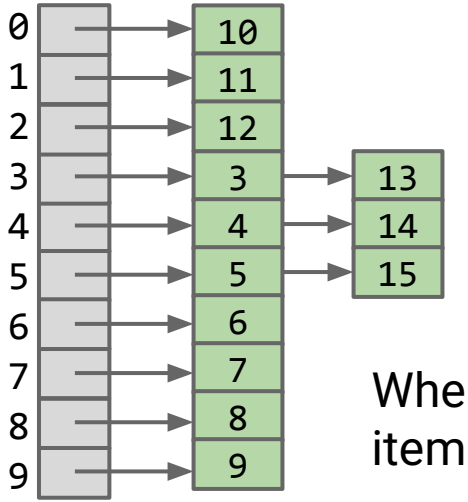
- How do we decide which integers go in which list? **We take each number mod 10.**
- Example: Suppose we have [3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], where would they go?



Even More Lists

Suppose we now have 10 lists.

- How do we decide which integers go in which list? **We take each number mod 10.**
- Example: Suppose we have [3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], where would they go?



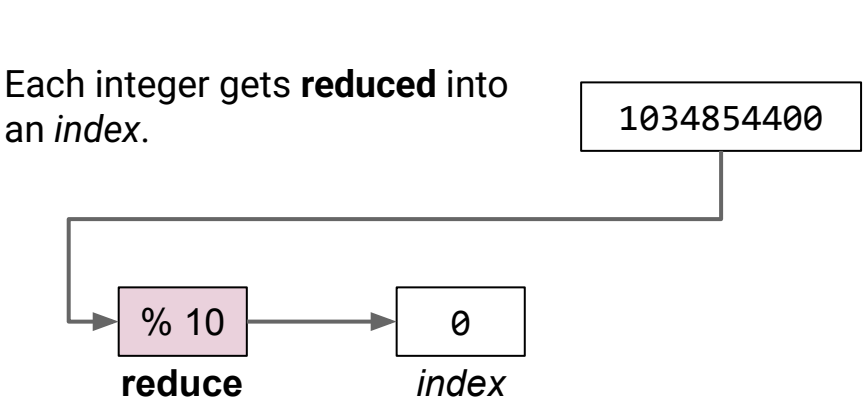
When a list contains multiple items, we call it a “collision”.

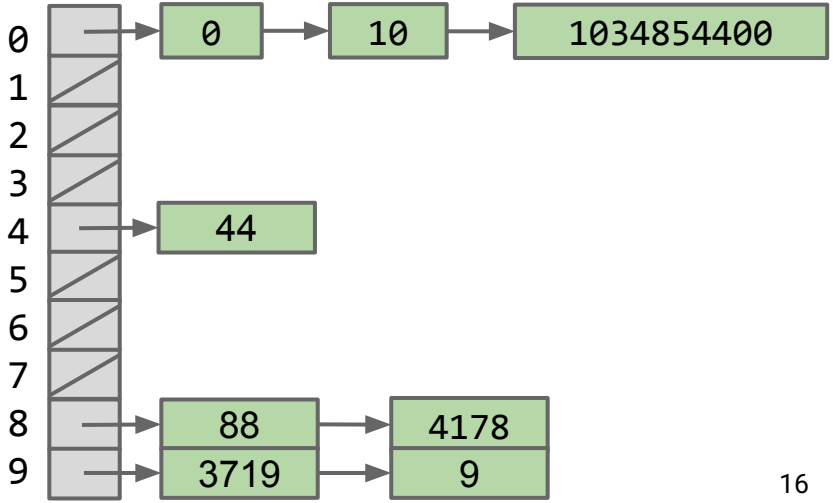
ArrayOfListsSet

This concept is generalizable to any number of lists. We will denote this number M. Below is a picture of our `ArrayOfListsSet` for M = 10.

If we have N items that are evenly distributed (minimizing the number of collisions), length of each list is about N/M.

- Results in an up to M fold speedup.
- If M = 10, all operations are up to 10 times faster.

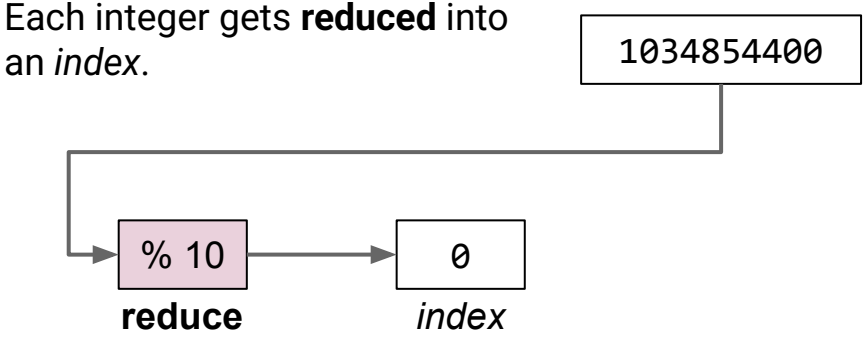


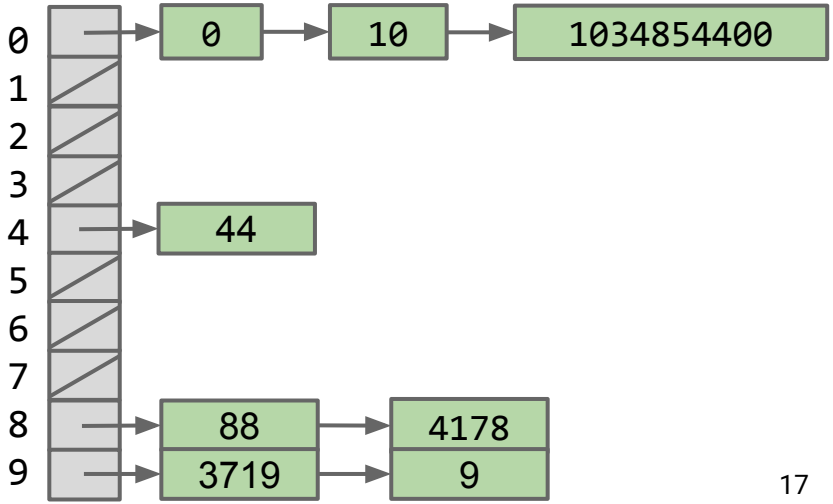


Buckets and Reduction Functions

For this type of data structure, each list is sometimes called a “bucket”.

- May have any number of items in a bucket.
- An integer lands in a bucket based on the reduction function.
  - **Modulus is the most natural and best reduction function**





DynamicArrayOfListsSet

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- **DynamicArrayOfListsSet**
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

Finding a Happy Medium

We've achieved a 10x speedup so far, which is pretty significant, but `contains` still takes linear time. Recall that Θ(N / 10) = Θ(N)

Finding a Happy Medium

We've achieved a 10x speedup so far, which is pretty significant, but `contains` still takes linear time. Recall that Θ(N / 10) = Θ(N)

Can we pick an M (number of buckets) that results in constant-time operations?

Finding a Happy Medium (Your Answer)

We've achieved a 10x speedup so far, which is pretty significant, but `contains` still takes linear time. Recall that Θ(N / 10) = Θ(N)

Can we pick an M (number of buckets) that results in constant-time operations?

What M should we pick?

- Idea: **make M depend on N**
  - Specifically, let's define M to be some **constant** multiple of N. This can be represented as $M = Nk$ where **k** is a constant.
  - Recall that contains runs in Θ(N / M). Now that M = Nk, we can substitute Nk for M in the asymptotic bound, resulting in Θ(1 / k) runtime!

Finding a Happy Medium (My Answer)

Note that our Θ(1 / k) runtime bound implies larger values of **k** result in a faster contains, which should align with our intuition about buckets, since more buckets results in a speedup, and **M** has a linear correlation with **k**.

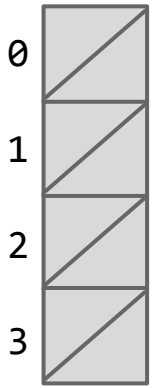
However, a larger k also results in more memory usage, so we'll pick a **k** that results in a happy medium.

Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

N = 0 M = 4 N / M = 0



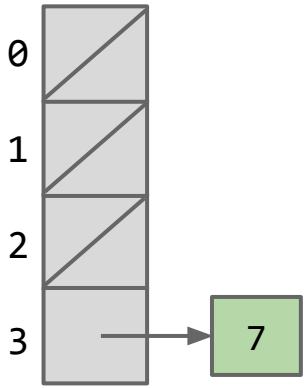
Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

- add(7)

N = 1 M = 4 N / M = 0.25

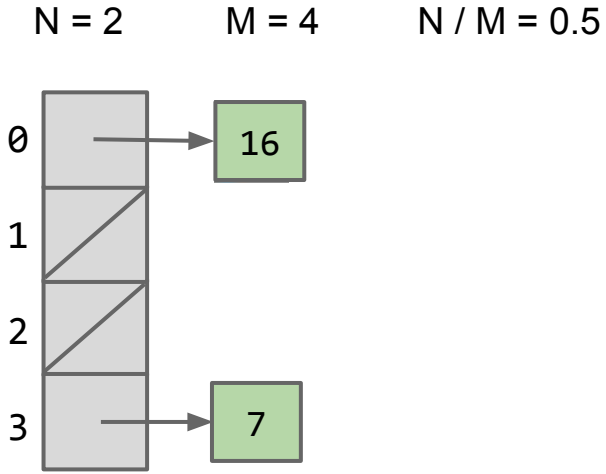


Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

- add(7), add(16)

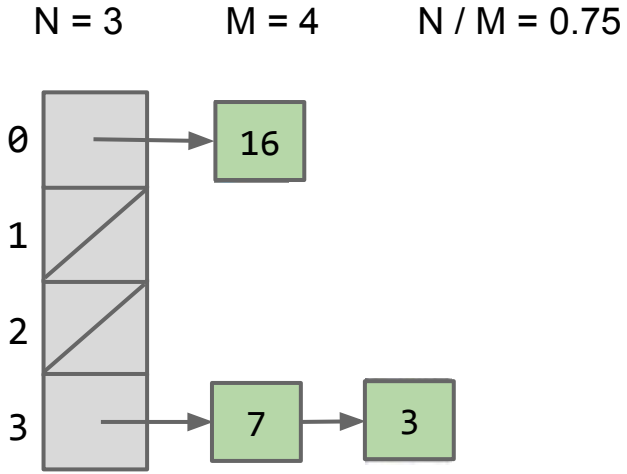


Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

- add(7), add(16), add(3)

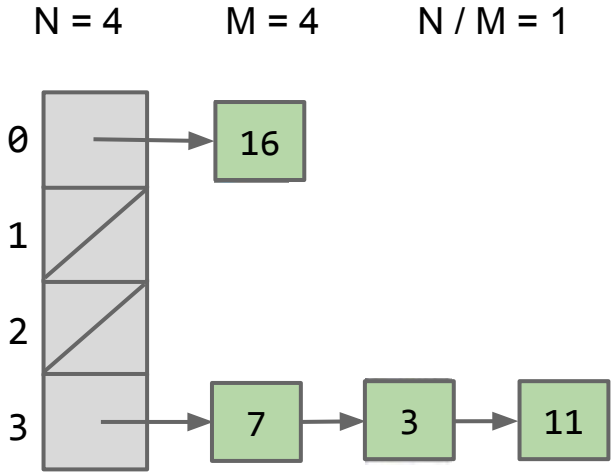


Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

- add(7), add(16), add(3), add(11)

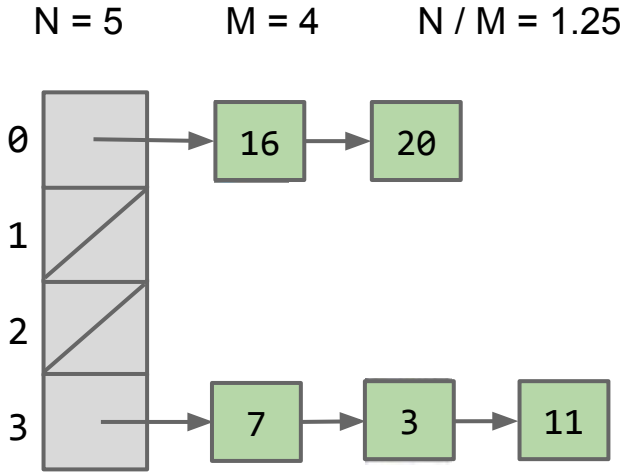


Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

- add(7), add(16), add(3), add(11), add(20)

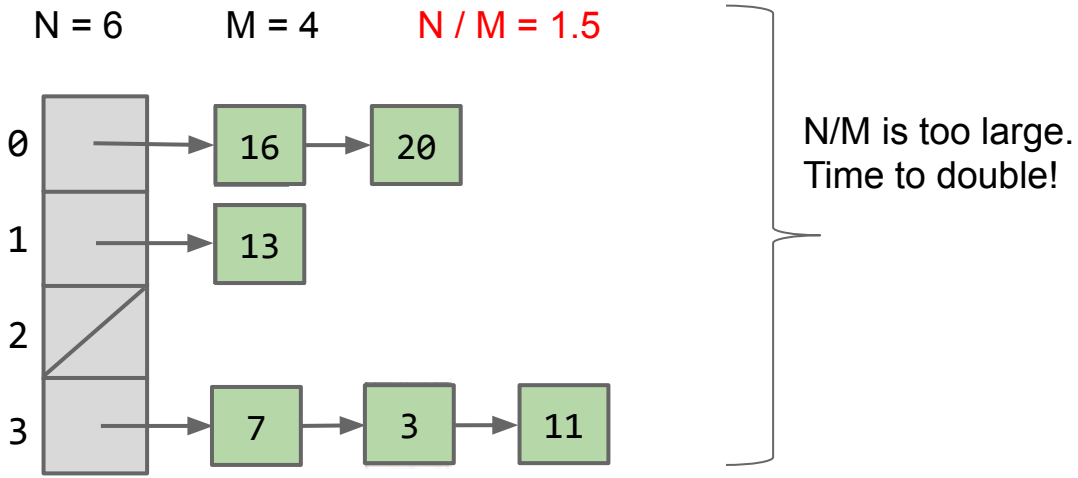


Hash Table Resizing Example

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

- add(7), add(16), add(3), add(11), add(20), add(13). Resize triggered.

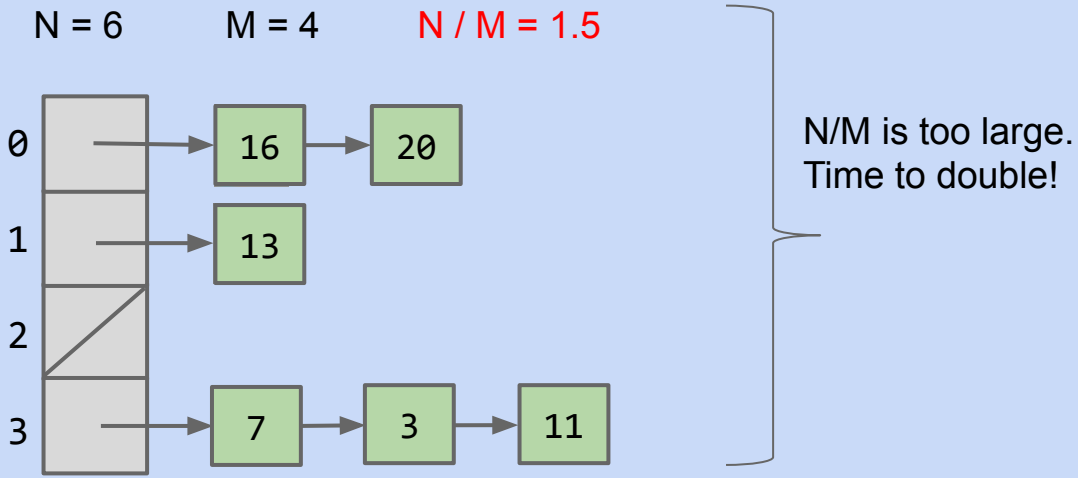


Hash Table Resizing Example: yellkey.com/else

Suppose we set **k** = 1.5

As such, when **N/M** is ≥ 1.5, we should double **M** to accomodate.

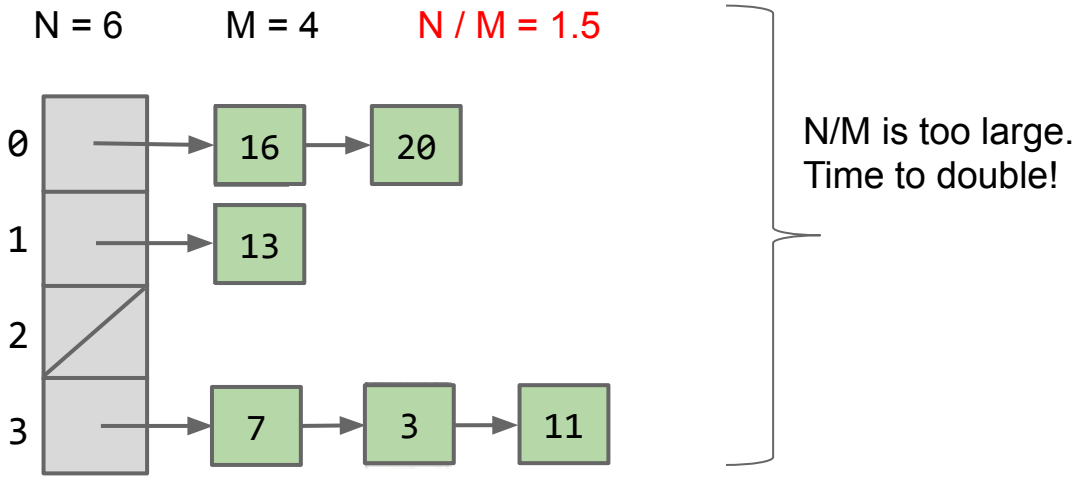
- Where will the 11 go?
- Or: Draw the results after doubling M.

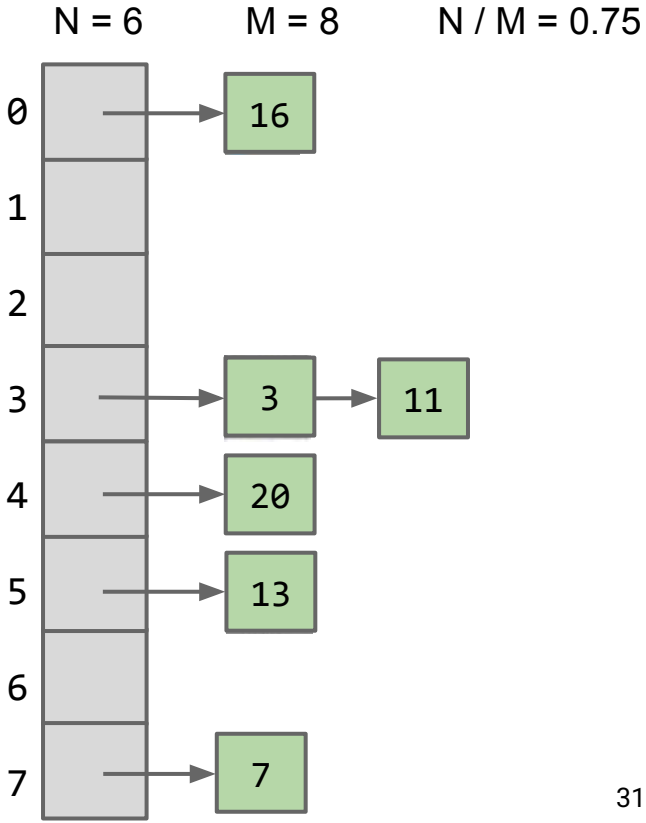


Hash Table Resizing Example

When N/M is ≥ 1.5, then double M.

- Draw the results after doubling M.





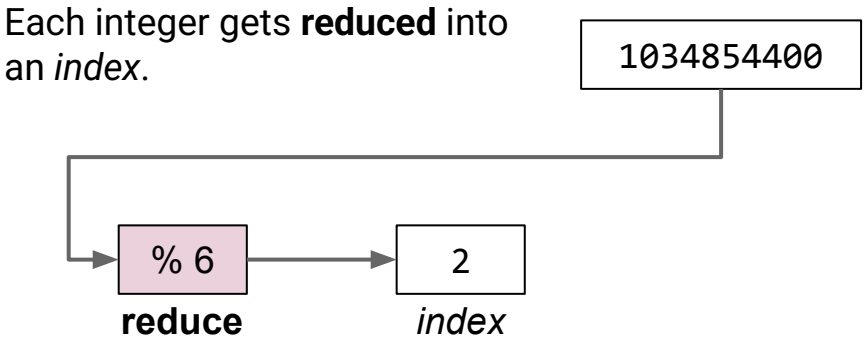
DynamicArrayOfListsSet

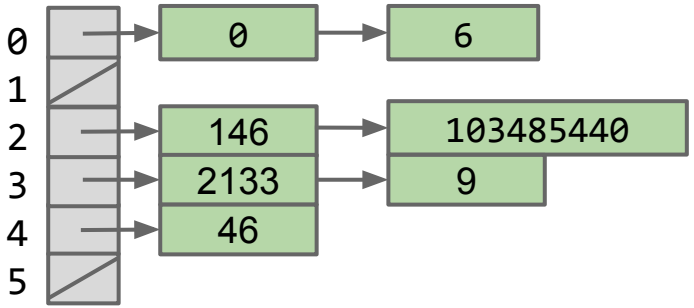
The data structure we just built might be called a `DynamicArrayOfListsSet`.

- Same as `ArrayOfListsSet`, but M increases as N increases by a factor of k.

If we have N items that are evenly distributed, length of each list is ~**k**.

- **k** is **constant** asymptotically.
- So operations are constant on average.





We'll think more carefully about runtime later.

lowercase strings

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- **lowercase strings**
- Integer Overflow
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

Goal: Storing Strings

The data structure we have so far is great for storing integers, because modulus on integers is intuitive.

Let’s try to figure out how to store Strings.

Storing the Word cat

Suppose we want to add("cat")

The key question:

- How do we reduce "cat" down to an integer?
- One idea: Use the order in the alphabet of the first letter as the list number, then apply modulus.
  - a = 1, b = 2, c = 3, …, z = 26

What are some issues with this approach?

Storing the Word cat (your answer)

Suppose we want to add("cat")

The key question:

- How do we reduce "cat" down to an integer?
- One idea: Use the order in the alphabet of the first letter as the list number, then apply modulus.
  - a = 1, b = 2, c = 3, …, z = 26
  - So cat would go in bucket 3.

What are some issues with this approach?

- …

Storing the Word cat (my answer)

Suppose we want to add("cat")

The key question:

- How do we reduce "cat" down to an integer?
- One idea: Use the order in the alphabet of the first letter as the list number, then apply modulus.
  - a = 1, b = 2, c = 3, …, z = 26
  - So cat would go in bucket 3.

What are some issues with this approach?

- Terrible worst cases - imagine a dataset full of words that all start with "a".
- Only the first 26 buckets ever see use.
- There are some strings that don't start with a letter, e.g. "1a2b3c"

Finding a Way to Store the Word cat

Suppose we want to add(“cat”)

The key question:

- How do we reduce “cat” down to an integer?

What is another idea? Assume for now we’re dealing with only lower case letters in English.

Finding a Way to Store the Word cat (Your Answer)

Suppose we want to add("cat")

The key question:

- How do we reduce "cat" down to an integer?

What is another idea? Assume for now we're dealing with only lower case letters in English.

- …

Finding a Way to Store the Word cat (My Answer)

Suppose we want to add("cat")

The key question:

- How do we reduce "cat" down to an integer?

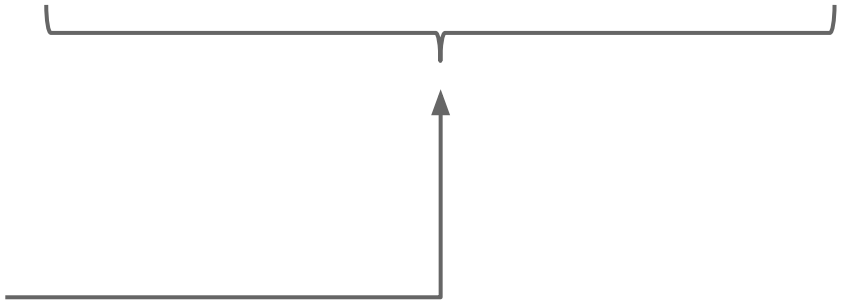
What is another idea? Assume for now we're dealing with only lower case letters in English.

- Treat cat as a base 26 number and just use the corresponding bucket.

Treating cat as a Base 26 Number

Use all digits by multiplying each by a power of 26.

- a = 1, b = 2, c = 3, …, z = 26
- Thus the index of “cat” is $(3 \times 26^{2}) + (1 \times 26^{1}) + (20 \times 26^{0}) = 2074$.



Why this specific pattern?

- Let’s review how numbers are represented in decimal.

Test Your Understanding

Convert the word “bee” into a number by using our “powers of 26” strategy.

Reminder: $cat_{26} = (3 \times 26^{2}) + (1 \times 26^{1}) + (20 \times 26^{0}) = 2074_{10}$

Hint: ‘b’ is letter 2, and ‘e’ is letter 5.

- $bee_{26} = (2 \times 26^{2}) + (5 \times 26^{1}) + (5 \times 26^{0}) = 1487_{10}$

The Decimal Number System vs. My System for Strings

In the decimal number system, we have 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Want numbers larger than 9? Use a sequence of digits.

Example: 7091 in base 10

- $7091_{10} = (7 \times 10^{3}) + (0 \times 10^{2}) + (9 \times 10^{1}) + (1 \times 10^{0})$

Our system for strings is almost the same, but with letters.

- One difference: In decimal numbers, 000 is the same as 0, but with strings aaa is different from a.
- To deal with this, we just don't have a 0 in our system, i.e. a is 1, not 0.

Test Your Understanding: yellkey.com/worker

Convert the word “bee” into a number by using our “powers of 26” strategy.

Reminder: $cat_{26} = (3 \times 26^{2}) + (1 \times 26^{1}) + (20 \times 26^{0}) = 2074_{10}$

Hint: ‘b’ is letter 2, and ‘e’ is letter 5.

Uniqueness

- $cat_{26} = (3 \times 26^{2}) + (1 \times 26^{1}) + (20 \times 26^{0}) = 2074_{10}$
- $bee_{26} = (2 \times 26^{2}) + (5 \times 26^{1}) + (5 \times 26^{0}) = 1487_{10}$

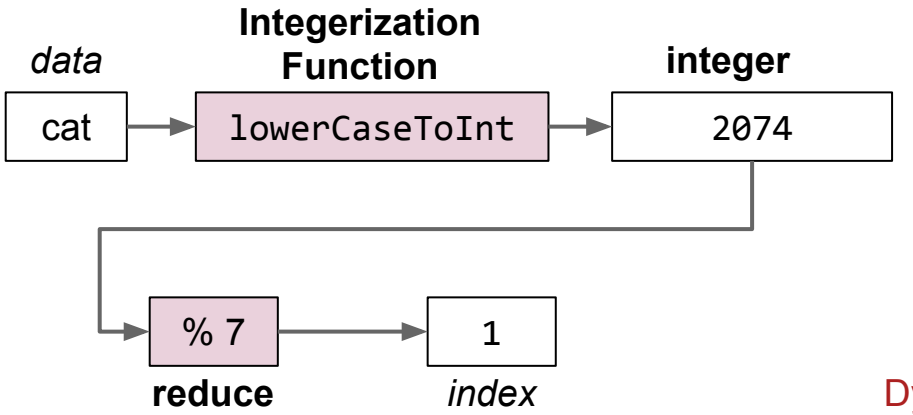
As long as we pick a base ≥ 26, this algorithm is guaranteed to give each lowercase English word a unique number!

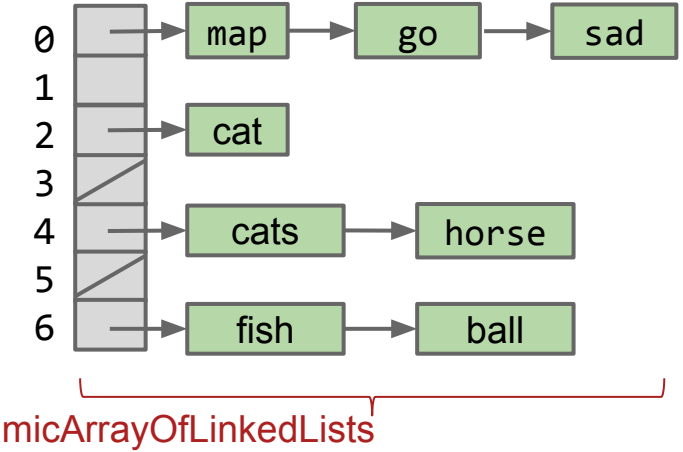
- Using base 26, no other words will get the number 1487.

DynamicArrayOfLinkedLists with Integerization

We've now extended our DynamicArrayOfLinkedLists to handle strings.

- *Data* is converted by a **integerization function** into an integer representation.
- The **integer** is then **reduced** to a valid *index*, usually using the modulus operator, e.g. 2348762878 % 10 = 8.





Implementing `englishToInt` (optional)

Optional exercise: Try to write a function `englishToInt` that can convert English strings to integers by adding characters scaled by powers of 26.

Examples:

- a: 1
- z: 26
- aa: 27
- bee: 1487
- cat: 2074
- dog: ??
- potato: ??

Implementing `englishToInt` (optional) (solution)

```
/** Converts ith character of String to a letter number.
    * e.g. 'a' -> 1, 'b' -> 2, 'z' -> 26 */
public static int letterNum(String s, int i) {
    int ithChar = s.charAt(i);
    if ((ithChar < 'a') || (ithChar > 'z'))
        { throw new IllegalArgumentException(); }
    return ithChar - 'a' + 1;
}

public static int englishToInt(String s) {
    int intRep = 0;
    for (int i = 0; i < s.length(); i += 1) {
        intRep = intRep * 26;
        intRep = intRep + letterNum(s, i);
    }
    return intRep;
}
```

Integer Overflow

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- **Integer Overflow**
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

DataIndexedStringSet

Using only lowercase English characters is too restrictive.

- What if we want to store strings like “2pac” or “eGg!”?

Suppose we wanted a unique integer for each possible such string.

- Need to assign an integer to all possible characters, e.g. what integer goes with “!”?

Someone has already done this.

- Let’s first discuss briefly discuss the ASCII standard.

ASCII Characters

The most basic character set used by most computers is ASCII format.

- Each possible character is assigned a value between 0 and 127.
- Characters 33 - 126 are “printable”, and are shown below.
- For example, `char c = 'D'` is equivalent to `char c = 68`.

Fun Fact: comparison in Java uses these values!
e.g. a = 97 < b = 98

33	!
34	"
35	#
36	$
37	%
38	&
39	'
40	(
41	)
42	*
43	+
44	,
45	-
46	.
47	/
48	0

49	1
50	2
51	3
52	4
53	5
54	6
55	7
56	8
57	9
58	:
59	;
60	<
61	=
62	>
63	?
64	@

65	A
66	B
67	C
68	D
69	E
70	F
71	G
72	H
73	I
74	J
75	K
76	L
77	M
78	N
79	O
80	P

81	Q
82	R
83	S
84	T
85	U
86	V
87	W
88	X
89	Y
90	Z
91	[
92	\
93	]
94	^
95	_
96	`

97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p

113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z
123	{
124	|
125	}
126	~

biggest value is 126

Maximum possible value for english-only text including punctuation is 126, so can use 126 as our base in order to ensure unique values for all possible strings.

Examples:

- $bee_{126} = (98 \times 126^2) + (101 \times 126^1) + (101 \times 126^0) = 1{,}568{,}675$
- $2pac_{126} = (50 \times 126^3) + (112 \times 126^2) + (97 \times 126^1) + (99 \times 126^0)$
  $= 101{,}809{,}233$
- $eGg!_{126} = (98 \times 126^3) + (71 \times 126^2) + (98 \times 126^1) + (33 \times 126^0)$
  $= 203{,}178{,}213$

Implementing `asciiToInt`

Below is a simple formula which converts a String to an integer.

- Treats String as a base 126 number.

```
public static int asciiToInt(String s) {
    int intRep = 0;
    for (int i = 0; i < s.length(); i += 1) {
        intRep = intRep * 126;
        intRep = intRep + s.charAt(i);
    }
    return intRep;
}
```

What if we want to use characters beyond ASCII?

Going Beyond ASCII

chars in Java also support character sets for other languages and symbols.

- `char c = '☔'` is equivalent to `char c = 9730`.
- `char c = '鳌'` is equivalent to `char c = 40140`.
- `char c = '혜'` is equivalent to `char c = 54812`.
- This encoding is known as Unicode. Table is too big to list.

Example: Computing Unique Representations of Chinese

The largest possible value for Chinese characters is 40,959*, so we'd need to use this as our base if we want to have a unique representation for all possible strings of Chinese characters.

Example:

- 守门员$_{40959}$ = (23432 x 40959^{2}) + (38376 x 40959^{1}) + (21592 x 40959^{0}) = 39,312,024,869,368

…	…
守门呗	39,3120,2486,9367
守门员	39,3120,2486,9368
守门务	39,3120,2486,9369
…	…

*If you're curious, the last character is: 鿕

Integer Overflow

Can we map every possible string to a unique integer?

- In Java, there are only a finite number of integers
- Exactly 4,294,967,296!
- Fixed precision*

That is, we tried to map 守门员 as a base 40959 number, yielding 39,312,024,869,368, but this number doesn't exist in Java as an int.

- How does Java represent this number then?

*Some programming languages such as Python support arbitrary precision, where the amount of numbers is limited by *memory* instead of *fixed* bits. In other words, integers can take on any value.

Integer Overflow

Can we map every possible string to a unique integer?

- In Java, the largest possible integer is 2,147,483,647.
- If you go over this limit, you overflow, starting back over at the smallest integer, which is -2,147,483,648.
- In other words, the next number after 2,147,483,647 is -2,147,483,648.

```
int x = 2147483647;
System.out.println(x);
System.out.println(x + 1);
```

```
jug ~/Dropbox/61b/lec/hashing
$ javac BiggestPlusOne.java
$ java BiggestPlusOne
2147483647
-2147483648
```

Integer Overflow

Can we map every possible string to a unique integer?

- Because Java has a maximum integer, we won't get the numbers we expect!
- With base 126, we will run into overflow even for short strings.
  - Example: $\text{omens}_{126} = 28{,}196{,}917{,}171$, which is much greater than the maximum integer!
  - `asciiToInt('omens')` will give us -1,867,853,901 in Java.

Hash Codes

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- **Hash Codes**

Hash Tables in Java

Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

Hash Codes

The official term for the number we're computing is "hash code".

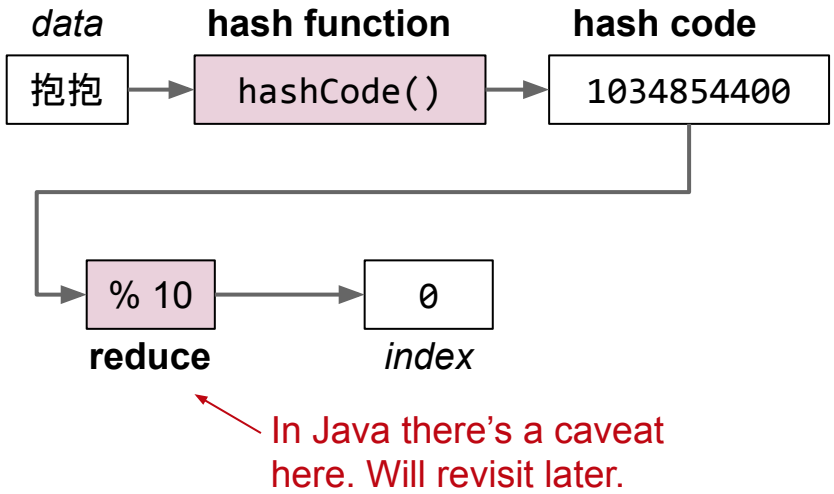
- Via Wolfram|Alpha: a hash code "projects a value from a set with many (or even an infinite number of) members to a value from a set with a fixed number of (fewer) members."
- Here, our target set is the set of Java integers, which is of size 4,294,967,296.

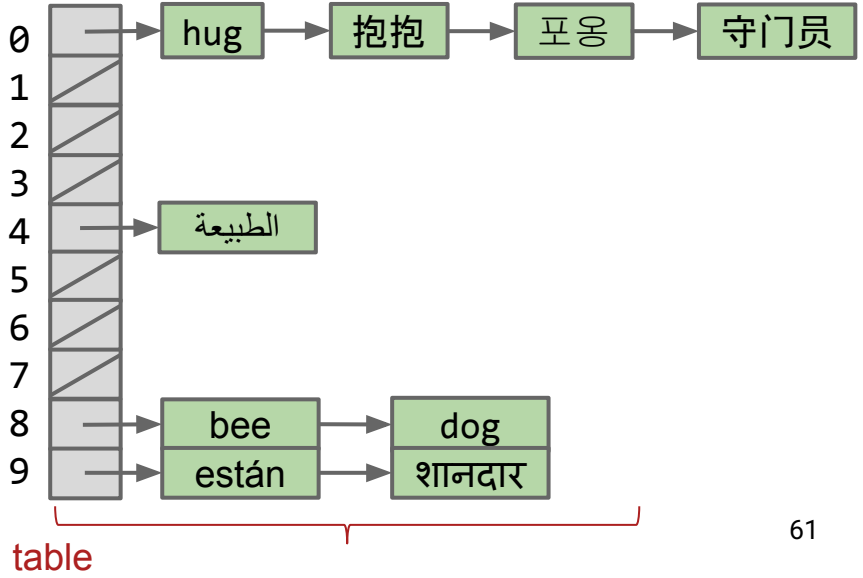
That is, our integerization function is a "hash code" because the set we're projected onto is fixed.

The Hash Table

A `DynamicArrayOfLists` with a finite integerization function is called a **hash table**.

- *Data* is converted by a **hash function** into a finite integer representation called a **hash code**. Java hash codes must be in [-2,147,483,648 to 2,147,483,647].
- The **hash code** is then **reduced** to a valid *index*, usually using the modulus operator, e.g. 2348762878 % 10 = 8.

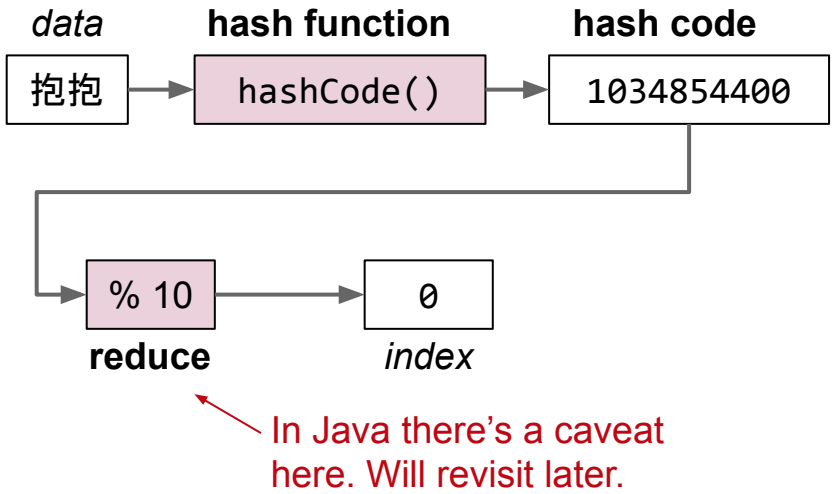


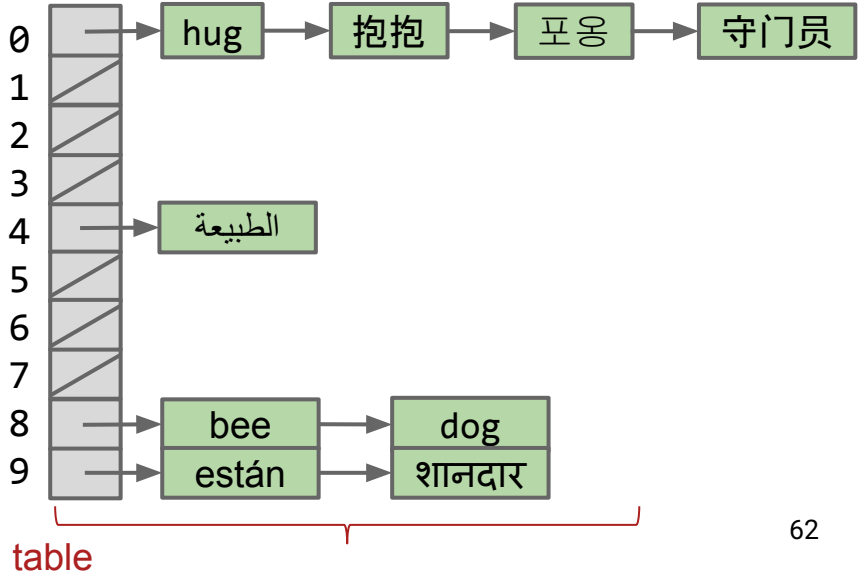


The Hash Table

Note, there are other versions of hash tables out there.

- The version we're using is an array of lists.
- This is sometimes called "separate chaining", where each bucket is a separate chain of items.
- Many more exotic solutions exist (linear probing, cuckoo hashing, using things other than linked lists for buckets, etc).





Hash Tables in Java

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

The Ubiquity of Hash Tables

Hash tables are the most popular implementation for sets and maps.

- Great performance in practice.
- Doesn't require items to be comparable.
- Implementations often relatively simple.
- Python dictionaries are just hash tables in disguise.

In Java, implemented as `java.util.HashMap` and `java.util.HashSet`.

- How does a HashMap know how to compute each object's hash code?
  - Good news: It's not "`implements Hashable`".
  - Instead, all objects in Java must implement a `.hashCode()` method.

Object Methods

All classes are hyponyms of `Object`.

- `String toString()`
- **`boolean equals(Object obj)`**
- `int hashCode()`
- `Class<?> getClass()`
- `protected Object clone()`
- `protected void finalize()`
- `void notify()`
- `void notifyAll()`
- `void wait()`
- `void wait(long timeout)`
- `void wait(long timeout, int nanos)`

From earlier in class.

This is where Java implements hash codes.

Won't discuss or use in 61B.

Default implementation of `hashCode` uses memory address.

Hash Codes in Java

Java's actual hashCode function for Strings below (code cleaned up slightly):

- "守门员" and "孬孍甭□□俎" map to 23,729,400.

```
public int hashCode(String s) {
    int intRep = 0;
    for (int i = 0; i < s.length(); i += 1) {
        intRep = intRep * 31;
        intRep = intRep + s.charAt(i);
    }
    return intRep;
}
```

Due to overflow, multiple strings have the same hashcode, e.g. the two calls below both return 23,729,400.

- "守门员".hashCode()
- "孬孍甭□□俎".hashCode()

More examples of Real Java HashCodes for Strings

```
System.out.println("a".hashCode());
System.out.println("bee".hashCode());
System.out.println("포옹".hashCode());
System.out.println("kamala lifefully".hashCode());
System.out.println("đậu hũ".hashCode());
```

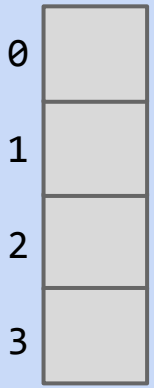
```
jug ~/Dropbox/61b/lec/hashing
$ java JavaHashCodeExamples
"a" 97
"bee" 97410
"포옹" 1732557
"kamala lifefully" 1732557
"đậu hũ" -2108180664
```

Using Negative hash codes

Suppose that 

's hash code is `-1`.

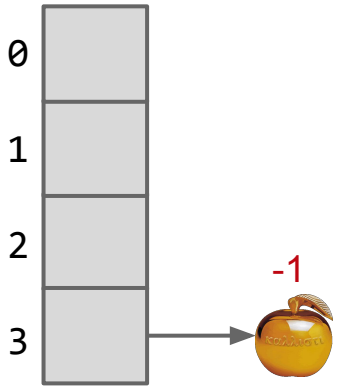
- Which bucket should we place this golden apple into?



Using Negative hash codes

Suppose that  ‘s hash code is `-1`.

- Which bucket should we place this golden apple into?
  - How about 3!
  - -1 loops back around to 3
  - Just like negative indexing in Python

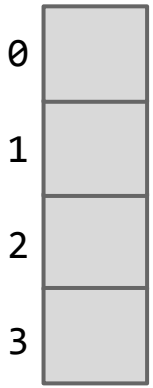


Using Negative hash codes in Java

Suppose that 

's hash code is `-1`.

- Unfortunately, -1 % 4 = -1. Will result in index errors!
- Use Math.floorMod instead.



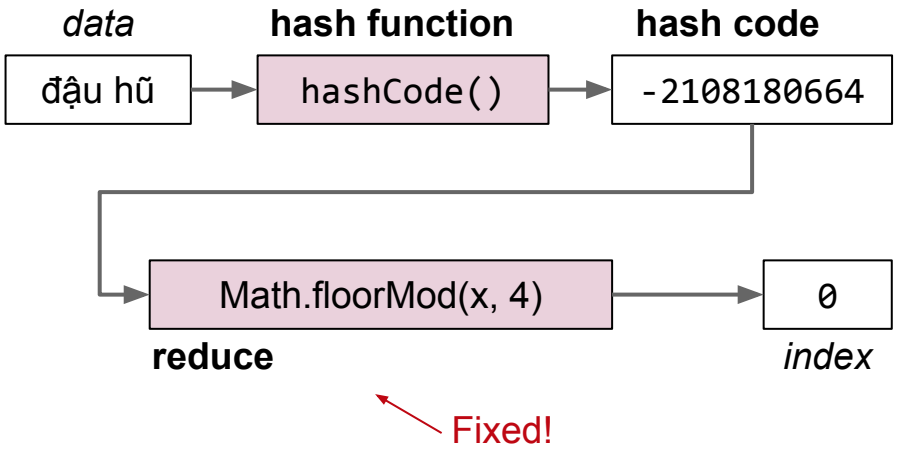

```
public class ModTest {
  public static void main(String[] args) {
    System.out.println(-1 % 4);
    System.out.println(Math.floorMod(-1, 4));
  }
}
```

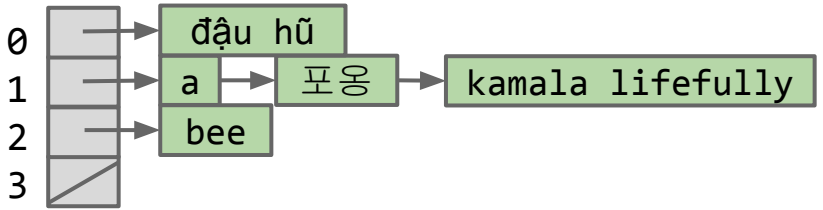
```
$ java ModTest
-1
3
```

Hash Tables in Java

Java hash tables:

- *Data* is converted by the **hashCode** method an integer representation called a **hash code**.
- The **hash code** is then **reduced** to a valid *index*, using something like the floorMod function, e.g. Math.floorMod(1732557 % 4) = 8.





Two Important Warnings When Using HashMaps/HashSets

Warning #1: Never store objects that can change in a `HashSet` or `HashMap`!

- Such objects are also called “mutable” objects, e.g. they can change.
  - Example: You’d never want to make a `HashSet<List<Integer>>`.
- If an object’s variables changes, then its hashCode changes. May result in items getting lost.

Warning #2: Never override `equals` without also overriding `hashCode`.

- Can also lead to items getting lost and generally weird behavior.
- HashMaps and HashSets use equals to determine if an item exists in a particular bucket.

We’ll come back to these warnings later.

Hash Table Performance and Summary

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

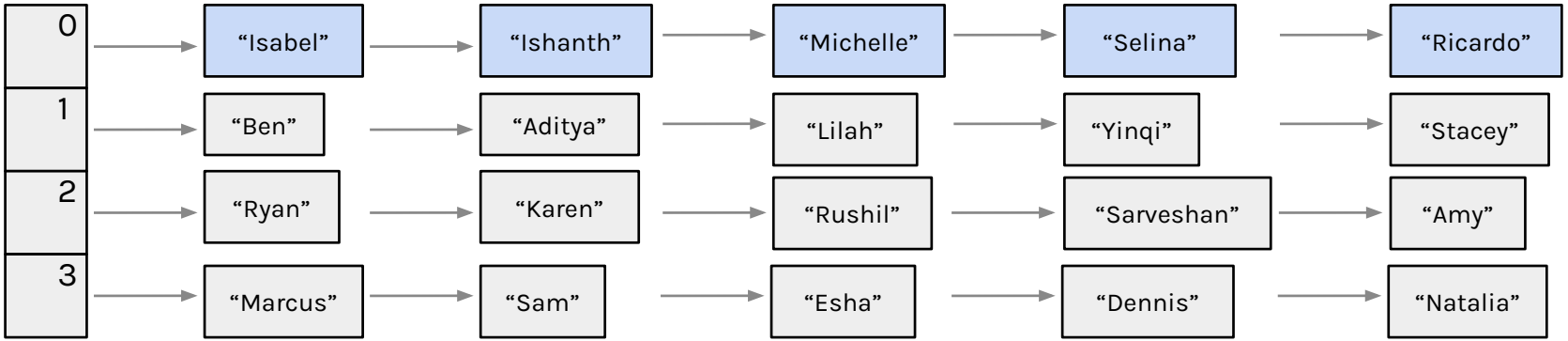
Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

Hash table with no resizing has Θ(N) runtimes

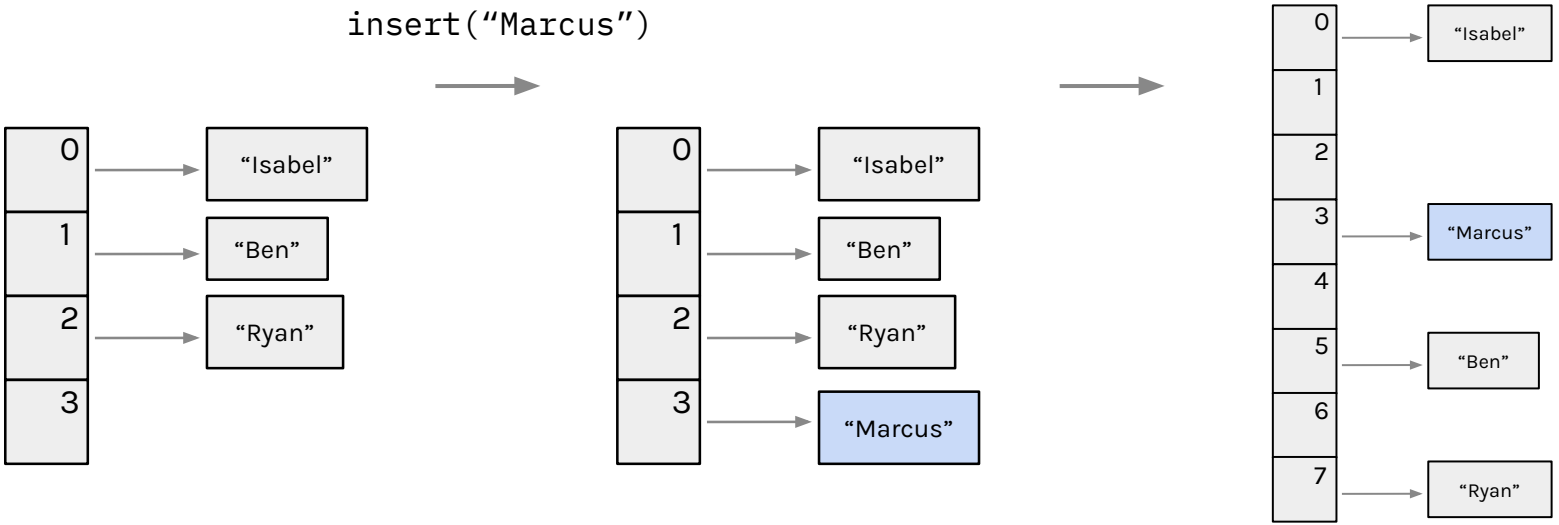
- The number of items N changes
- The number of buckets M *does not and stays constant*
- Therefore, buckets contain…
  - Θ(N/M) → Θ(N)* items if we have an even distribution



*Treat M as a constant to simplify, e.g. if M = 4 then Θ(N/4) → Θ(N)

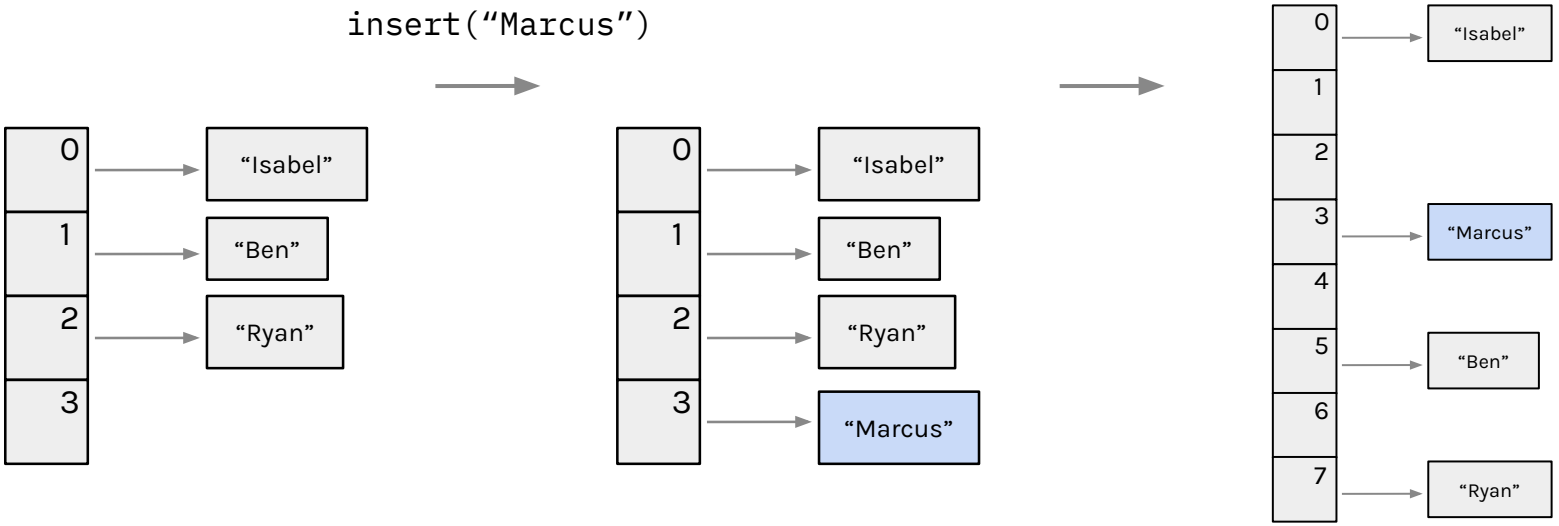
Hash table with resizing on average has Θ(1) runtimes

- The number of items N changes
- The number of buckets M *changes as N changes*
- Therefore, buckets contain…
  - Θ(N/M) → Θ(1) items if we have an even distribution



Hash table with resizing on average has Θ(1) runtimes

- Why on average?
  - What happens if we need to resize? Θ(N)
  - What happens if we don't have an even distribution? Θ(N)



What contributes to our Θ(N) resize runtime?

What contributes to our Θ(N) resize runtime?

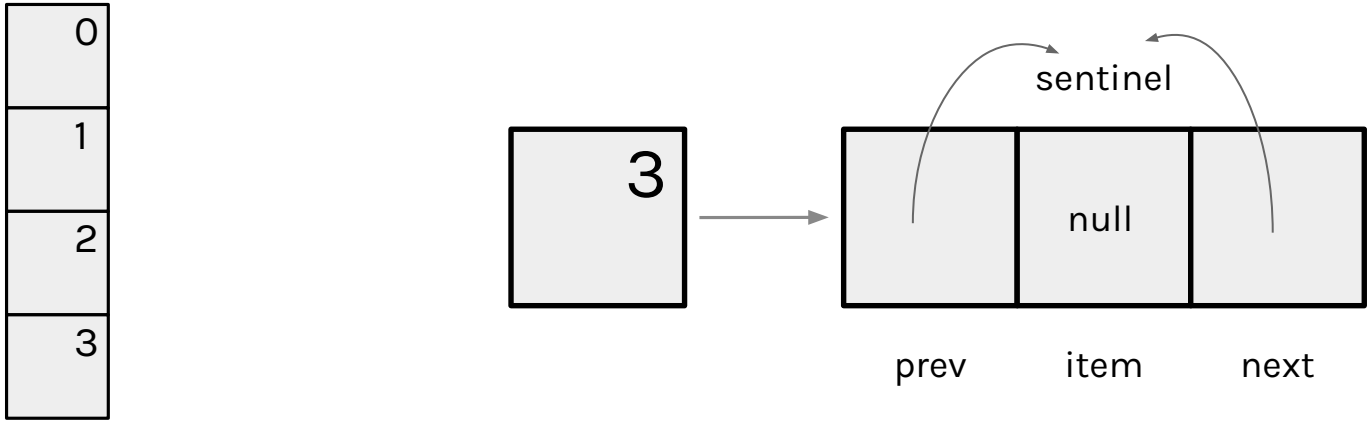
- Double number of buckets → Θ(M)
- Iterate through M buckets → Θ(M)
- Iterate through N items → Θ(N)
  - For each, re-calculate its hashcode and its new index → Θ(1)
  - For each, re-insert into the buckets → Θ(1)*

Since M = Θ(N), the total runtime is Θ(M) + Θ(M) + Θ(N) * (Θ(1) + Θ(1)) = Θ(N)

*Θ(1) time assuming insertion at a *known* index of the linked list such as head insertion (see next slides).

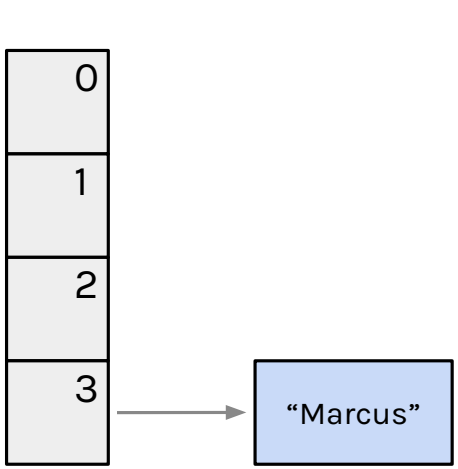
What contributes to our Θ(N) resize runtime?

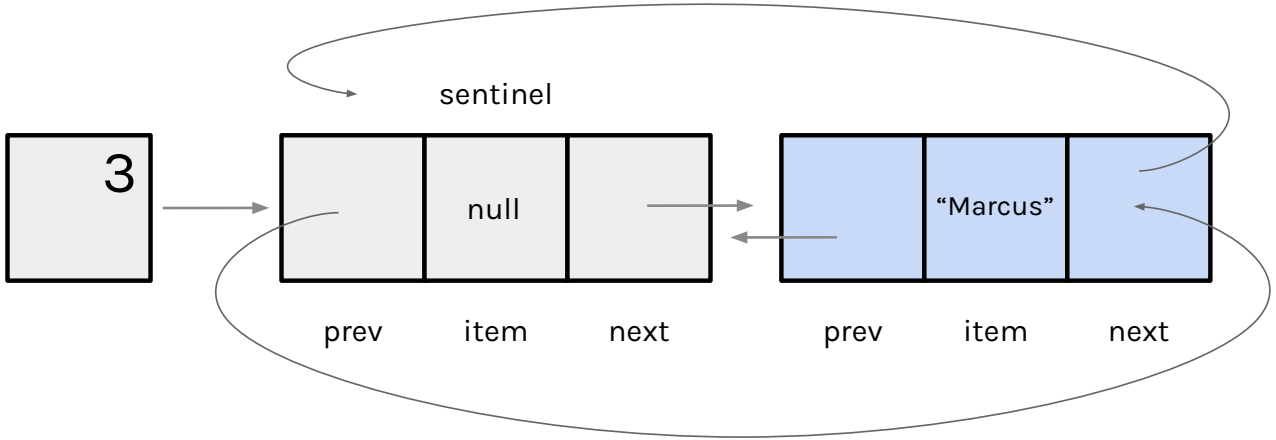
Head Insertion Example



What contributes to our Θ(N) resize runtime?

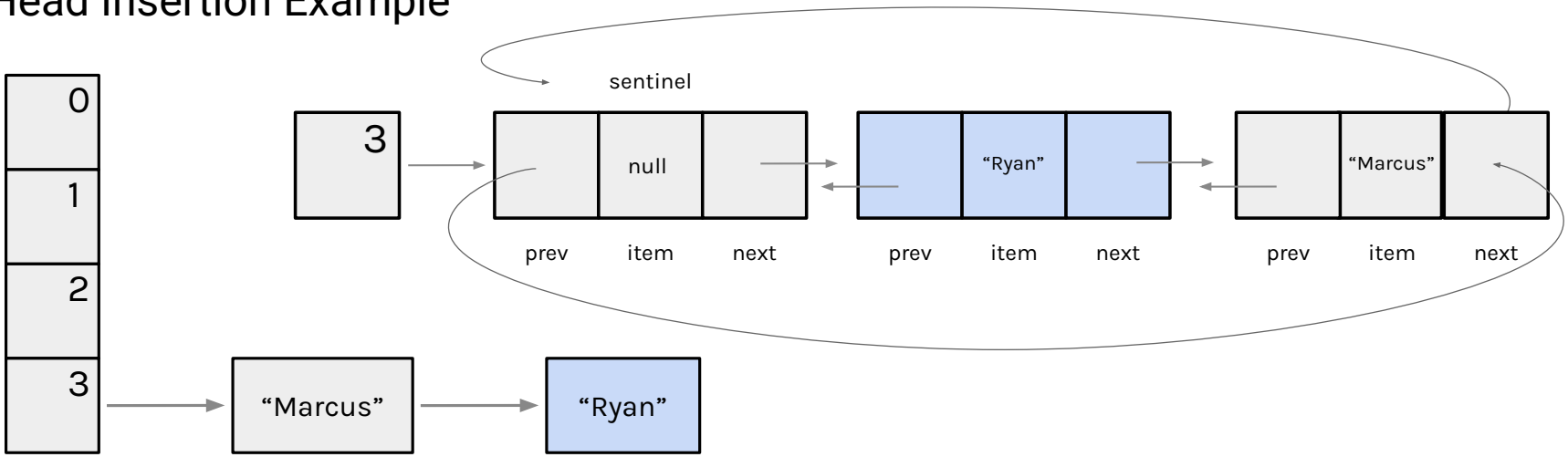
Head Insertion Example





What contributes to our Θ(N) resize runtime?

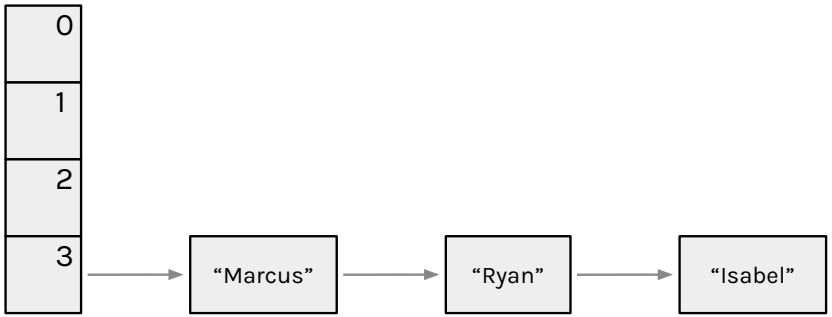
Head Insertion Example*

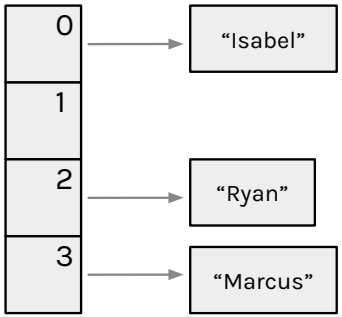


*For out of scope reasons, you can ignore how separate chaining is done in practice, but this is a good example of what could be happening under the hood. Can you prove to yourself this takes Θ(1) time? In reality, Java uses a mix of tail insertion and red-black trees, more on that in Hashing II!

Even distribution of items ensures good performance

- Both tables below have the same load factor of N/M = 0.75.
- However, the left table has worse performance for `contains(String x)`!
- Θ(N)* vs Θ(1) respectively

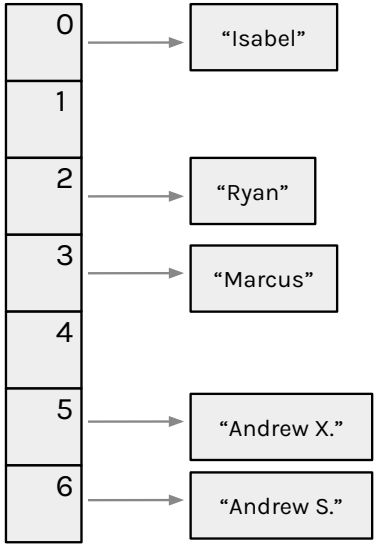




*Remember that we need to access each element in the bucket, e.g. linked list traversal takes Θ(N) time.

How do we get an even distribution of items?

- `index = hashcode % buckets.length`
  - Hash Code
  - Table Size (number of buckets)
- See Creating a Good Hash Code (extra)



Summary

Insertion

- Check if the element already exists and insert the new element if not
- Check the load factor (usually 0.75) and resize if needed
  - Load factor = N/M or items/buckets

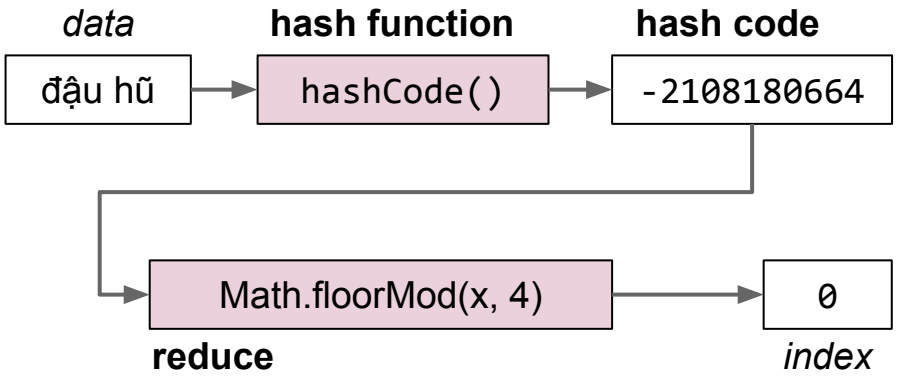
Resize

- Increase the backing array's* size (usually 2x)
- Iterate through all items in the *original* backing array
  - For each, calculate it's new index using its hash code and the *new* backing array length
  - Then, insert the item into the *new* backing array

*Buckets are the same thing as the backing array!

Hash tables

- *Data* is converted into a hash code.
- The hash code is then reduced to a valid *index*.
- *Data* is then stored in a bucket corresponding to that *index*.
- Resize when load factor N/M exceeds some constant.
- If items are spread out nicely, you get Θ(1) average runtime.



	`contains(x)`	`add(x)`
Bushy BSTs	Θ(log N)	Θ(log N)
Separate Chaining Hash Table With No Resizing	Θ(N)	Θ(N)†
… With Resizing	Θ(1)†	Θ(1)*†

*: Indicates “on average”.
†: Assuming items are evenly spread.

Creating a Good Hash Code (extra)

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

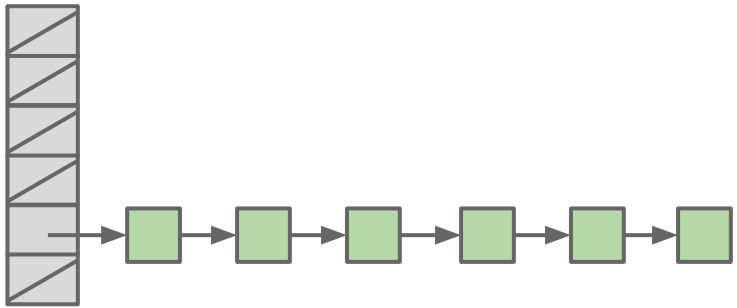
Creating a Good Hash Code (extra)

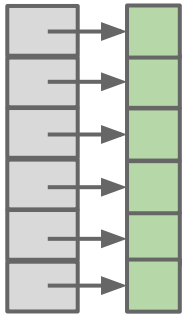
Linear Probing (extra)

What Makes a good .hashCode()?

Goal: We want hash tables that look like the table on the right.

- Want a hashCode that spreads things out nicely on real data.
  - Example #1: return 0 is a bad hashCode function.
  - Example #2: just returning the first character of a word, e.g. “cat” → 3 was also a bad hash function.
  - Example #3: Adding chars together is bad. “ab” collides with “ba”.
  - Example #4: returning string treated as a base B number can be good.
- Writing a good hashCode() method **can be tricky**.





Hashbrowns and Hash Codes

How do you make hashbrowns?

- Chopping a potato into nice predictable segments? No way!
- Similarly, adding up the characters is not nearly “random” enough.

Can think of multiplying data by powers of some base as ensuring that all the data gets scrambled together into a seemingly random integer.

Example hashCode Function

The Java 8 hash code for strings. Two major differences from our hash codes:

- Represents strings as a base 31 number.
  - Why such a small base? Real hash codes don't care about uniqueness.
- Stores (caches) calculated hash code so future `hashCode` calls are faster.

```
@Override
public int hashCode() {
    int h = cachedHashValue;
    if (h == 0 && this.length() > 0) {
        for (int i = 0; i < this.length(); i++) {
            h = 31 * h + this.charAt(i);
        }
        cachedHashValue = h;
    }
    return h;
}
```

Example: Choosing a Base

Java's `hashCode()` function for Strings:

- $h(s) = s_0 \times 31^{n-1} + s_1 \times 31^{n-2} + \ldots + s_{n-1}$

Our `asciiToInt` function for Strings:

- $h(s) = s_0 \times 126^{n-1} + s_1 \times 126^{n-2} + \ldots + s_{n-1}$

Which is better?

- Might seem like 126 is better. Ignoring overflow, this ensures a unique numerical representation for all ASCII strings.
- … but overflow is a particularly bad problem for base 126!

Example: Base 126

Major collision problem:

- “geocronite is the best thing on the earth.”.hashCode() yields 634199182.
- “flan is the best thing on the earth.”.hashCode() yields 634199182.
- “treachery is the best thing on the earth.”.hashCode() yields 634199182.
- “Brazil is the best thing on the earth.”.hashCode() yields 634199182.

Any string that ends in the same last 32 characters has the same hash code.

- Why? Because of overflow.
- Basic issue is that 126^32 = 126^33 = 126^34 = ... 0.
  - Thus upper characters are all multiplied by zero.
  - See CS61C for more.

Typical Base

A typical hash code base is a small prime.

- Why prime?
  - Never even: Avoids the overflow issue on previous slide.
  - Lower chance of resulting hashCode having a bad relationship with the number of buckets
- Why small?
  - Lower cost to compute.

A full treatment of good hash codes is well beyond the scope of our class.

Hashbrowns and Hash Codes

How do you make hashbrowns?

- Chopping a potato into nice predictable segments? No way!

Using a prime base yields better “randomness” than using something like base 126.

Example: Hashing a Collection

Lists are a lot like strings: Collection of items each with its own hashCode:

```
@Override
public int hashCode() {
    int hashCode = 1;
    for (Object o : this) {
        hashCode = hashCode * 31;
        hashCode = hashCode + o.hashCode();
    }
    return hashCode;
}
```

elevate/smear the current hash code

add new item's hash code

To save time hashing: Look at only first few items.

- Higher chance of collisions but things will still work.

Example: Hashing a Recursive Data Structure

Computation of the hashCode of a recursive data structure involves recursive computation.

- For example, binary tree hashCode (assuming sentinel leaves):

```
@Override
public int hashCode() {
    if (this.value == null) {
        return 0;
    }
    return  this.value.hashCode() +
     31 * this.left.hashCode() +
     31 * 31 * this.right.hashCode();
}
```

Linear Probing (extra)

Lecture 26, CS61B, Spring 2026

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- DynamicArrayOfListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Tables in Java

Hash Table Performance and Summary

Creating a Good Hash Code (extra)

Linear Probing (extra)

Open Addressing: An Alternate Disambiguation Strategy (Extra)

Instead of using linked lists, an alternate and more exotic strategy is “open addressing”.

- Set is stored as an array of items. Index tells you where to put the item.

If target location is already occupied, use a different location, e.g.

- Linear probing: Use next address, and if already occupied, just keep scanning one by one.
  - Demo: http://goo.gl/o5EDvb
- Quadratic probing: Use next address, and if already occupied, try looking 4 ahead, then 9 ahead, then 16 ahead, …
- Many more possibilities. See the optional reading for today (or CS170) for a more detailed look.

In 61B, we’ll use the “separate chaining” approach, where we have linked lists.

Citations

http://www.nydailynews.com/news/national/couple-calls-911-forgotten-mcdonalds-hash-browns-article-1.1543096

http://en.wikipedia.org/wiki/Pigeonhole_principle#mediaviewer/File:TooManyPigeons.jpg

https://cookingplanit.com/public/uploads/inventory/hashbrown_1366322674.jpg

FAQ

What is the distinction between hash set, hash map, and hash table?

A hash set is an implementation of the Set ADT using the “hash table” as its engine.

A hash map is an implementation of the Map ADT using the “hash table” as its engine.

A “hash table” is a way of storing information, where you have M buckets that store N items. Each item has a “hashCode” that tells you which of M buckets to put that item in.